

CS496 AI / Generative AI Tutorials - Solved with Explanations

Covers Tutorial 2.1, 2.2, 2.3, and 2.4 from the uploaded tutorial zip

Goal: exam-ready answers that are easy to memorize but technically correct. Compound questions are answered fully under the same question number.

Files covered: Tutorial 2.1 Generative AI; Tutorial 2.2 Underlying Deep Learning technologies; Tutorial 2.3 LangChain; Tutorial 2.4 Autonomous Agent Systems.

Important correction: The tutorial/slide wording sometimes says activation functions control learning rate. For the exam, know the course wording, but the technically correct explanation is that activation functions add non-linearity and shape neuron outputs/gradients; the optimizer learning-rate hyperparameter controls update step size.

Coverage Checklist

Section	Major question items	Status
Tutorial 2.1 - Generative AI	12	Solved
Tutorial 2.2 - Underlying Deep Learning Technologies	13	Solved
Tutorial 2.3 - LangChain	10	Solved
Tutorial 2.4 - Autonomous Agent Systems	12	Solved

Total major question items solved: 47.

Tutorial 2.1 - Generative AI

Q1. What is Generative AI? What is the main goal of Generative AI?

Generative AI is a type of AI that learns patterns from existing data and uses those patterns to create new content. The generated content can be text, images, audio, video, code, molecules, designs, or other data.

The main goal is not just classification or prediction. The goal is to model the data distribution well enough to generate new outputs that look valid, useful, and similar in style or structure to the training data, while not simply copying one training example.

Exam key: Generative AI learns patterns from data and generates new, realistic content such as text, images, audio, code, or video.

Q2. What are the major types of models used to implement Generative AI?

The main model families in the tutorial are sequence models, GANs, VAEs, and transformers. Modern systems may also use diffusion models for images/video, but the course tutorial focuses mainly on the four families below.

Model type	Core idea	Typical use
Sequential RNN/LSTM models	Process data step by step and remember previous context through hidden states.	Text or sequence generation.
GANs	A generator creates fake samples and a discriminator judges real vs fake.	Realistic image/audio/data generation.
VAEs	Encode data into a probabilistic latent space, then decode samples from that space.	Generate variations similar to training data.
Transformers	Use attention to model relationships between tokens and generate context-aware outputs.	LLMs such as GPT, summarization, translation, coding.

Exam key: Know LSTM/RNN sequence models, GANs, VAEs, and transformers. Diffusion is important generally, but not central in this tutorial set.

Q3. Describe the general architecture of Sequence-to-Sequence models and explain how they work.

A sequence-to-sequence model maps an input sequence to an output sequence. The classic architecture has two parts: an encoder and a decoder.

The encoder reads the input sequence and compresses its meaning into an internal representation, often called a context vector or hidden state. The decoder then uses that representation to generate the output sequence one token at a time. This allows input and output lengths to be different, which is useful for machine translation, summarization, and chatbot responses.

Exam key: Seq2Seq = encoder reads input -> context/state -> decoder generates output step by step.

Q4. Describe the general architecture of GAN models and explain how they work.

A Generative Adversarial Network has two neural networks trained against each other.

The generator takes random noise and tries to produce fake samples that look real. The discriminator receives both real samples and generated fake samples and tries to classify which is which. Training is adversarial: the discriminator improves at detecting fake samples, while the generator improves at fooling the discriminator. At the end, a good generator can produce realistic new samples.

Exam key: GAN = generator vs discriminator. Generator creates fake data; discriminator distinguishes real/fake; both improve through adversarial training.

Q5. Describe the general architecture of Variational Autoencoder models and explain how they work.

A Variational Autoencoder has an encoder, a latent space, and a decoder. The encoder maps input data to a probability distribution in latent space, usually represented by a mean and variance. The decoder samples from that latent space and reconstructs or generates data.

The model learns a smooth probabilistic representation of the data. After training, sampling from the latent space can create new examples with similar characteristics to the training set.

Exam key: VAE = probabilistic encoder -> latent space with mean/variance -> decoder generates or reconstructs samples.

Q6. Describe the general architecture of the Transformer model and explain how it works.

A Transformer processes tokens using attention instead of relying on step-by-step recurrence. Input text is split into tokens, converted to embeddings, combined with positional information, and passed through stacked Transformer blocks.

The core operation is self-attention. Self-attention lets each token compare itself with other tokens in the sequence and decide which tokens matter most for understanding context. Multi-head attention repeats this process in several learned ways at the same time. Feed-forward layers then transform the representations. This is why transformers handle long-range dependencies better than older RNN/LSTM models.

The original Transformer uses an encoder-decoder design. GPT-style models mainly use decoder-only Transformer blocks with masked self-attention, so they predict the next token using previous tokens only.

Exam key: Transformer = token embeddings + positional information + self-attention + feed-forward blocks. GPT is decoder-only and predicts next tokens.

Q7. What is the GPT deep learning model?

GPT stands for Generative Pre-trained Transformer. It is generative because it can produce new text, pre-trained because it is first trained on large amounts of text before task-specific adaptation, and transformer-based because it uses the Transformer attention architecture.

GPT models are commonly used as large language models. They predict and generate token sequences and can be adapted to tasks such as answering questions, summarizing, translating, coding, and reasoning-like problem solving.

Exam key: GPT = Generative Pre-trained Transformer; a transformer-based language model trained to generate text/token sequences.

Q8. How is GPT pre-trained? What approach is used?

GPT is pre-trained mainly with self-supervised autoregressive language modeling. The training data is plain text, and the model creates its own learning signal from the sequence.

In GPT-style training, the model receives previous tokens and learns to predict the next token. Example: from "The cat sat on the", the target is likely "mat" depending on the text. Repeating this over massive text corpora teaches grammar, syntax, semantics, facts, styles, and patterns of reasoning.

Exam key: GPT pre-training = self-supervised next-token prediction/autoregressive modeling.

Q9. What is fine-tuning and how does it work?

Fine-tuning adapts a pre-trained model to a narrower task, domain, style, or behavior. Instead of training from zero, we start with a model that already learned general language patterns, then continue training it on a smaller specialized dataset.

For example, a general model can be fine-tuned on medical, legal, programming, or customer-service data. Fine-tuning updates model parameters so the model performs better on the target domain. In modern LLM systems, instruction tuning and preference alignment are also used to make the model follow instructions and behave more usefully.

Exam key: Fine-tuning = continue training a pre-trained model on specialized data to adapt it to a task/domain.

Q10. What are Large Language Models (LLMs)?

Large Language Models are deep learning models, usually transformer-based, trained on very large text/token datasets to understand and generate language. They estimate the probability of token sequences and use that ability to answer questions, summarize, translate, write code, classify text, and perform multi-step language tasks.

They are called large because they usually have many parameters and are trained on very large corpora. Their strength is flexible language generation; their weakness is that they can hallucinate, have limited or outdated knowledge unless connected to tools/retrieval, and do not automatically have persistent memory.

Exam key: LLM = large transformer-based language model trained on huge text datasets to understand and generate human language.

Q11. What is ChatGPT?

ChatGPT is an AI chatbot by OpenAI that uses large language models to understand prompts and generate conversational responses. In course language, it can act as a reasoning engine for tasks such as interpreting goals, planning steps, writing code, answering questions, using tools when tools are connected, and supporting multi-step workflows.

Do not describe ChatGPT as a database or as guaranteed truth. It generates responses based on learned patterns and available context. Its accuracy improves when it is grounded with reliable sources, tools, retrieval systems, or course material.

Exam key: ChatGPT = OpenAI conversational LLM chatbot; useful as a reasoning/generation engine, but not automatically a guaranteed source of truth.

Q12. ChatGPT is initially pre-trained using self-supervised learning. What is self-supervised learning, and how does it differ from common deep learning approaches?

Self-supervised learning trains on unlabeled data by creating labels automatically from the data itself. For text, the model can hide or shift parts of the sequence and learn to predict them. GPT-style models use next-token prediction: the input is previous tokens, and the label is the next token already present in the text.

The difference from common supervised learning is that supervised learning uses human-provided labels, such as cat/dog labels or spam/not-spam labels. Self-supervised learning does not require manual labels at large scale; the structure of the data creates the training target.

Exam key: Self-supervised learning uses unlabeled data but creates targets from the data itself; GPT uses next-token prediction.

Tutorial 2.2 - Underlying Deep Learning Technologies

Q13. Briefly describe the evolution of Artificial Intelligence (AI) over the years.

AI began in the 1950s with the goal of automating intellectual tasks normally associated with humans. Early AI relied heavily on symbolic reasoning and manually written rules.

In the 1970s and 1980s, expert systems became common. They used large rule bases but were brittle and narrow. In the 1990s and 2000s, classical machine learning became more important: instead of hand-writing all rules, systems learned patterns from examples. In the 2010s, deep learning became dominant for perception tasks such as image recognition, speech recognition, translation, and recommendation. In the 2020s, transformers and LLMs added strong language generation, few-shot learning, and agentic workflows. Current agentic AI systems combine LLMs with tools, memory, retrieval, planning, and feedback loops.

Exam key: AI evolved from symbolic rules -> expert systems -> machine learning -> deep learning -> transformers/LLMs -> autonomous AI agents.

Q14. Describe the general structure of a typical Multilayer Perceptron (MLP). How are such models trained? What are the roles of weights?

A Multilayer Perceptron is a feed-forward neural network made of layers of neurons. It has an input layer, one or more hidden layers, and an output layer.

The input layer receives features. Hidden layers compute transformations of those features. The output layer produces the final prediction, such as a class probability or a numeric value.

Training works by forward propagation, loss calculation, backpropagation, and optimization. First, input data moves forward through the network. The prediction is compared with the desired output using a loss function. Backpropagation computes how each weight and bias contributed to the error. An optimizer such as gradient descent or stochastic gradient descent updates the weights and biases to reduce the loss.

Weights control the strength and direction of connections between neurons. Biases shift activation thresholds. Together, weights and biases store the learned rules of the model.

Exam key: MLP = input layer + hidden layer(s) + output layer. Training = forward pass -> loss -> backpropagation -> weight/bias updates.

Q15. What are activation functions and what purpose are they used for?

An activation function transforms a neuron's weighted input into its output. The neuron first computes a weighted sum plus bias, then applies the activation function.

The main purpose is to add non-linearity. Without non-linear activation functions, multiple layers would collapse into one linear transformation and the network could not learn complex patterns. Activation functions also shape the output range and influence gradient flow during training.

Accuracy note: The tutorial/slide wording says activation functions control learning rate. Technically, the optimizer learning-rate hyperparameter controls the step size. Activation functions mainly add non-linearity and shape neuron outputs/gradients.

Exam key: Activation function = nonlinear transformation applied after weighted sum; enables MLPs to learn complex patterns.

Q16. Provide examples of activation functions and explain how they work.

Common activation functions include sigmoid, tanh, ReLU, leaky ReLU, and softmax.

Function	Formula / output	How it works / use
Sigmoid	$1 / (1 + e^{-z})$, output 0 to 1	Squashes values into a probability-like range; used in binary outputs or older hidden layers.
Tanh	-1 to 1	Centers output around zero; useful but can still suffer from vanishing gradients.
ReLU	$\max(0, z)$	Outputs zero for negative values and z for positive values; common in hidden layers.
Leaky ReLU	z if $z > 0$, small $\cdot z$ otherwise	Keeps a small gradient for negative values to reduce dead neurons.
Softmax	Converts logits into class probabilities that sum to 1	Used in multi-class classification output layers.

Exam key: Know sigmoid from the MLP example: $f(z) = 1/(1+e^{-z})$. Also know ReLU and softmax basics.

Q17. What are loss functions and what purpose are they used for?

A loss function measures how wrong the model's prediction is compared with the target output. It converts prediction error into a number that the model tries to minimize.

Loss functions guide learning. During training, backpropagation computes gradients of the loss with respect to weights and biases. The optimizer updates the parameters in the direction that reduces the loss.

Exam key: Loss function = error measure between predicted output and target; training minimizes it by updating weights and biases.

Q18. What are the loss and activation functions used in the provided MLP examples?

In the course MLP example, the explicit activation function shown is the sigmoid/logistic function:

$$f(z) = 1 / (1 + \exp(-z))$$

The slides discuss the loss function generally as the difference between the target value and the actual computed value. The gradient descent example treats the loss/cost as a curve to minimize. It does not clearly name one single MLP loss such as MSE or cross-entropy.

A correct exam-safe answer is: the example uses a sigmoid activation function, and a generic cost/loss function measuring prediction error. For a simple numeric/regression MLP, this is often represented as mean squared error. For classification, common losses are binary cross-entropy or categorical cross-entropy.

Exam key: Course-specific: sigmoid activation; loss = target-vs-output error minimized by gradient descent. Do not invent a named loss unless the question asks generally.

Q19. How is the output of each node in the MLP computed?

Each neuron computes two steps.

Step 1: calculate weighted sum plus bias:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

Step 2: apply activation function:

$$h = f(z)$$

The value h becomes that neuron's output and is passed to the next layer. In vector form for a layer:

$$h = f(Wx + b)$$

Exam key: Neuron output = activation(weighted sum + bias), $h = f(Wx + b)$.

Q20. How do MLPs learn? What is the commonly used algorithm, and how does it work?

MLPs learn by adjusting weights and biases to reduce loss on training examples. The commonly used algorithm is backpropagation combined with an optimizer such as gradient descent or stochastic gradient descent.

The process is: perform a forward pass to compute a prediction, calculate the loss, backpropagate the error from output layer to earlier layers, compute gradients for each weight and bias, and update parameters in the direction that reduces the loss. Repeating this over many examples makes the network gradually learn useful input-output relationships.

Exam key: MLP learning = forward pass -> loss -> backpropagation -> gradient descent/SGD update -> repeat.

Q21. Describe the backpropagation algorithm and explain how it works.

Backpropagation is the algorithm that calculates how much each parameter contributed to the final error. It uses the chain rule from calculus to propagate gradients backward from the output layer through the hidden layers.

First, the network predicts an output. Second, the loss function measures the error. Third, the derivative of the loss is calculated with respect to output-layer weights, then previous hidden-layer weights, and so on. Finally, an optimizer uses those gradients to update weights and biases. The goal is to minimize the loss over time.

Exam key: Backpropagation sends error gradients backward through the network using the chain rule, allowing weights/biases to be updated.

Q22. What are the roles of activation and loss functions in the MLP learning process?

The activation function controls how each neuron transforms its input and introduces non-linearity, allowing the MLP to learn complex relationships. It also affects the flow of gradients during backpropagation.

The loss function measures the difference between the prediction and the desired output. It provides the training objective: minimize this error. Without a loss function, the model would not know whether its prediction is good or bad.

Accuracy note: For technical correctness: activation functions do not set the optimizer learning rate. They influence neuron outputs and gradients. The learning rate is a separate optimizer parameter.

Exam key: Activation gives nonlinear neuron output; loss measures prediction error and drives weight updates.

Q23. Which MLP parameters encapsulate the hidden learning rules used to predict or classify inputs in production mode?

The learned rules are encoded mainly in the weights and biases of every layer. During training, backpropagation and optimization modify these parameters. After training, the final weight matrices and bias vectors represent the learned mapping from inputs to outputs.

In production, the architecture and activation functions are also needed, but the learned knowledge itself is stored in the trained numerical parameters: W and b .

Exam key: The trained weights W and biases b store the learned rules.

Q24. Explain how the MLP model can be used in production mode after training.

After training, the model is used in inference mode. The trained weights and biases are saved. For a new input, the model performs only a forward pass through the layers to generate a prediction.

No backpropagation happens in normal production inference. The model does not update its parameters unless it is retrained or fine-tuned later. Production use is therefore: load model -> preprocess input -> forward pass -> output prediction/classification.

Exam key: Production/inference mode = use saved W and b for forward pass only; no gradient updates.

Q25. Explain in detail how the Transformer deep learning model works: architecture, loss functions, accuracy, and self-supervised learning.

A Transformer begins by converting text into tokens. Tokens are mapped to embeddings, and positional information is added so the model knows token order. The embeddings pass through stacked Transformer blocks.

Each block contains multi-head self-attention and feed-forward layers, usually with normalization and residual connections. Self-attention computes which tokens should pay attention to which other tokens. Multi-head attention learns several relationship patterns in parallel. Feed-forward networks then transform each token representation.

Architecture: the original Transformer has an encoder and decoder. The encoder builds contextual representations of the input. The decoder generates output while using masked attention so it cannot look at future tokens. GPT-style LLMs usually use decoder-only blocks and predict the next token.

Loss function: for language modeling, the standard loss is cross-entropy between the predicted token probability distribution and the correct next token.

Accuracy/evaluation: for pre-training, perplexity or validation cross-entropy is commonly used; lower perplexity means the model predicts tokens better. For fine-tuned classification tasks, accuracy can be measured as correct predictions divided by total predictions.

Self-supervised learning: the labels come from the data itself. GPT-style models use next-token prediction. BERT-style models use masked-token prediction. This allows training on massive unlabeled text corpora.

Exam key: Transformer = embeddings + positional info + self-attention + feed-forward blocks. GPT uses decoder-only masked self-attention and cross-entropy next-token loss.

Tutorial 2.3 - LangChain

Q26. What is LangChain? What is the main motivation behind building LangChain?

LangChain is a framework for building applications powered by language models. Its motivation is that a raw LLM alone is not enough for many real applications.

A normal LLM can generate text, but it may not remember long conversations, access private documents, call tools, search databases, or reliably execute multi-step workflows. LangChain provides building blocks such as prompts, models, tools, chains, retrieval, vector stores, memory, and agents so developers can connect LLMs to external data and actions.

Exam key: LangChain = framework for LLM applications; adds retrieval, tools, memory, chains, and agents around the LLM.

Q27. What are the major problems with Large Language Models?

Major LLM problems include: no persistent memory by default, knowledge limited by training data unless connected to external sources, hallucination, limited context window, no direct access to private data or live systems, weak reliability in long multi-step tasks, and no ability to take real actions unless tools/APIs are integrated.

Exam key: LLM weaknesses: no built-in persistent memory, outdated/limited knowledge, hallucinations, no external access, context limits, and weak tool/action ability alone.

Q28. What are the solutions provided by LangChain to solve these problems?

LangChain addresses these issues by wrapping the LLM in application components. Retrieval-Augmented Generation connects the LLM to external/private data. Vector stores and retrievers search relevant document chunks. Memory keeps conversation state or summaries. Tools allow API calls, calculations, search, database access, and custom functions. Chains/runnables organize multi-step workflows. Agents let the LLM decide which tool or step to use next. Prompt templates make inputs consistent and reusable.

Exam key: LangChain solutions: RAG/retrieval, vector stores, memory, tools, chains/runnables, agents, and prompt templates.

Q29. Use diagrams to show the difference between a conventional LLM such as ChatGPT and a LangChain model/application.

A conventional LLM workflow is simple: the user sends a prompt and the model returns a response. A LangChain application adds supporting components around the model so the response can be grounded, stateful, and tool-based.

Conventional LLM:

User Prompt -> LLM -> Response

LangChain-style application:

User Input
-> Prompt Template
-> Memory / Conversation State
-> Retriever / Vector Store / External Data
-> LLM
-> Agent decides tool use if needed
-> Tools / APIs / DB / Search
-> Final Response

Exam key: LangChain is not a new LLM by itself; it is an application framework around LLMs.

Q30. What are the major components of LangChain?

Course-level major components include models, prompts, memory, indexes, retrievers, vector stores, chains, agents, and tools. In newer LangChain terminology, workflows are often built with runnables and LangGraph, but the same basic ideas remain: connect models to context, steps, state, and actions.

Component	Role
Models	LLM or chat model used for generation/reasoning.
Prompts	Structured templates that control input format.
Memory	Stores conversation or state across interactions.
Indexes	Organize external documents/data for search.
Retrievers	Fetch relevant chunks for a query.
Vector stores	Store embeddings for semantic similarity search.
Chains / Runnables	Connect steps into a workflow.
Agents	Use an LLM to decide actions/tools dynamically.
Tools	Functions/APIs/search/calculators/databases the agent can call.

Exam key: Know: models, prompts, memory, indexes/retrievers/vector stores, chains, agents, tools.

Q31. Explain how LangChain mitigates the problems of LLMs.

LangChain maps each weakness to a supporting component.

LLM problem	LangChain mitigation
No persistent state	Memory stores conversation history, summaries, or application state.
Outdated/private knowledge unavailable	Retrievers and vector stores implement RAG over documents/databases.
Hallucination	Ground answers in retrieved evidence and constrain prompts.

No external actions	Tools let the model call APIs, search, calculators, DBs, or custom functions.
Weak multi-step flow	Chains/runnables and agents structure the workflow.
Inconsistent prompting	Prompt templates standardize instructions and variables.

Exam key: Memory fixes statelessness; RAG fixes missing knowledge; tools fix no action; chains/agents fix multi-step workflows.

Q32. What is Retrieval Augmentation and how does it work?

Retrieval Augmentation, commonly called Retrieval-Augmented Generation or RAG, improves an LLM by giving it relevant external information at answer time.

The workflow is: collect documents, split them into chunks, convert chunks into embeddings, store embeddings in a vector store, receive a user query, embed the query, retrieve the most relevant chunks, insert those chunks into the prompt, and let the LLM answer using that context. This makes the answer more grounded and reduces hallucination.

Exam key: RAG = retrieve relevant external chunks and inject them into the prompt before generation.

Q33. Using a simple example, demonstrate how LangChain may be used to fix LLM problems.

Example: a student asks, "What does my CS496 tutorial say about LangChain memory?" A raw LLM may guess because it does not know the private tutorial file. With LangChain, the application loads the tutorial, splits it into chunks, embeds the chunks, stores them in a vector database, retrieves the relevant memory section, and sends that section to the LLM as context. The answer is then based on the actual document instead of guessing.

This fixes missing private knowledge, reduces hallucination, and lets the same app support follow-up questions by using memory.

Exam key: Simple LangChain fix: use RAG over private documents so the LLM answers from real course content.

Q34. What are the roles of Memory, Indexes, Retrievers, Vector Stores, Prompts, Chains, Agents, and Tools in solving a LangChain problem?

Each component solves a specific part of the application problem.

Component	Role in a LangChain problem
Memory	Keeps conversation context, user preferences, or previous results.
Indexes	Prepare and organize external content for retrieval.
Retrievers	Select the most relevant content for the current query.
Vector stores	Store embeddings and perform semantic similarity search.
Prompts	Format instructions, context, and variables sent to the LLM.
Chains	Run fixed multi-step pipelines such as retrieve -> answer -> summarize.
Agents	Choose actions/tools dynamically based on the goal.
Tools	Execute external actions such as search, calculator, database query, email, APIs.

Exam key: Memorize the mapping: memory=context, retriever=fetch, vector store=semantic DB, prompts=format, chains=steps, agents=decide, tools=act.

Q35. Write a complete pseudocode LangChain example application that solves a typical LLM problem.

Below is pseudocode for a course-document Q&A app using RAG plus memory. It is intentionally pseudocode so the idea stays correct even if exact LangChain imports change.

```
# Goal: answer questions from private course/tutorial documents

load documents from folder
split documents into small chunks
create embeddings for each chunk
store chunks + embeddings in vector_store
create retriever from vector_store

create chat_model
create prompt_template:
    system: "Answer only from the provided context. If missing, say you do not know."
    context: retrieved_chunks
    chat_history: previous_messages
    question: user_question

create memory to store conversation history

function answer_question(user_question):
    relevant_chunks = retriever.search(user_question)
    prompt = prompt_template.format(
        context=relevant_chunks,
        chat_history=memory.load(),
        question=user_question
    )
    response = chat_model.invoke(prompt)
    memory.save(user_question, response)
    return response
```



```
# Optional agent version:  
tools = [retriever_tool, calculator_tool, web_search_tool]  
agent receives user question  
agent decides which tool to call  
agent observes tool result  
agent produces final answer
```

Exam key: A complete LangChain app usually has documents -> chunks -> embeddings -> vector store -> retriever -> prompt -> LLM -> memory/final answer.

Tutorial 2.4 - Autonomous Agent Systems

Q36. What are the major distinctive features between traditional software and Agentic AI in behavior and problem-solving approach?

Traditional software is usually deterministic and rule-based. Developers explicitly write the steps, conditions, and outputs. The system waits for input, executes predefined logic, and returns a predictable result. It is good for stable workflows with clear rules. Agentic AI is goal-directed and adaptive. The developer gives the agent a goal, tools, memory, policies, and environment access. The agent can perceive the situation, reason about context, plan steps, choose tools, act, observe results, and revise its strategy. It is useful when the problem is dynamic, uncertain, language-heavy, or requires multiple steps.

Aspect	Traditional software	Agentic AI
Control	Developer hard-codes the flow.	Agent chooses actions toward a goal.
Behavior	Mostly deterministic.	Adaptive, context-dependent, probabilistic.
Problem solving	Fixed algorithms and rules.	Perception, reasoning, planning, acting, learning.
Memory	Usually explicit database/session state.	Persistent working/semantic/episodic memory can guide behavior.
External tools	Called only where programmer coded them.	Agent can select tools dynamically.
Failure handling	Predefined exception paths.	Can observe failures, re-plan, retry, or ask for help.

Exam key: Traditional = fixed instructions. Agentic = autonomous goal pursuit using perception, reasoning, planning, action, memory, and learning.

Q37. What is an autonomous AI Agent?

An autonomous AI agent is an AI system that can pursue a goal by repeatedly observing its environment, reasoning about what the observation means, planning actions, executing actions through tools or APIs, and learning from outcomes.

It is not just a chatbot response. An agent has an objective, state/memory, access to tools, and a control loop that lets it take multi-step actions with some independence.

Exam key: Autonomous AI agent = goal-driven system that perceives, reasons, plans, acts, and learns using memory and tools.

Q38. What is the main difference between traditional software architectures and AI Agentic architectures? Describe using simple diagrams the architecture of AI agents.

Traditional architecture is usually a fixed pipeline. Agentic architecture is a feedback loop with memory and tools.

Traditional software architecture:

Input -> Predefined program logic -> Database/API if coded -> Output

AI agent architecture:

Goal/User Input
-> Perception module
-> Reasoning module
-> Planning module
-> Action / Tool-use module
-> Observation of result
-> Learning / Memory update
-> back to Perception/Reasoning until goal is done

Supporting systems:

- Memory: working, short-term, long-term, semantic, episodic, procedural
- Tools: APIs, search, databases, calculators, code execution, apps
- Policies/guardrails: safety, permissions, constraints

Exam key: Agentic architecture is loop-based and stateful; traditional architecture is fixed-flow and predefined.

Q39. Describe the AI Agentic Cognitive Loop.

The cognitive loop is the repeated cycle an agent uses to operate autonomously:

1. Perception: capture input or environment data.
2. Reasoning: interpret the input and infer meaning, intent, priority, and constraints.
3. Planning: break the goal into ordered steps and choose a strategy.
4. Action: execute steps using tools, APIs, databases, or communication.
5. Learning: evaluate outcome, store useful experience, and update memory/strategy.

The loop repeats because the result of an action becomes new information to perceive. Memory supports every phase by storing context and past experience.

Exam key: Cognitive loop = Perceive -> Reason -> Plan -> Act -> Learn -> repeat.

Q40. Using a simple example, how is perception performed in AI agents? What are the requirements?

Perception means collecting raw information from the environment and converting it into a structured form the agent can use.

Example: a customer-service agent receives the message: "My payment failed and I need help now." Perception extracts the user message, user ID, time, channel, sentiment, intent clues, and any attached transaction ID.

Requirements include input sources such as user text, APIs, sensors, files, databases, or tool outputs; parsing/extraction logic; structured representation of the observation; and access permissions. Good perception must preserve relevant context because later reasoning depends on it.

Exam key: Perception = gather data from user/API/environment and structure it for reasoning.

Q41. Using a simple example, explain how reasoning is performed in AI agents.

Reasoning means interpreting the perceived data and deciding what it implies.

Example: after perceiving "My payment failed and I need help now," the agent classifies the intent as a billing/payment issue, detects negative sentiment and urgency, checks memory for recent payment attempts, and concludes that the next step should be to verify the transaction status.

Reasoning can use LLM inference, rules, classifiers, retrieval, past memory, or statistical models. The output is an understanding of the situation, such as intent, priority, likely cause, missing information, and confidence.

Exam key: Reasoning = convert perception into understanding: intent, meaning, priority, cause, confidence, and next needs.

Q42. Using a simple example, explain how planning is undertaken by AI agents. What is the outcome of the planning phase?

Planning turns a goal into an ordered sequence of steps.

Example task: "Book a flight." The agent understands the goal, then decomposes it into steps: ask for travel dates and destination if missing, search flights, compare price/time/stops, select the best option based on user preferences, enter user details, request confirmation, and complete booking if authorized.

The outcome of planning is a structured action plan: ordered actions, required tools, needed information, expected results, and stopping conditions.

Exam key: Planning = break a goal into ordered executable steps. Output = action plan ready for execution.

Q43. Using a simple example, explain how decisions and actions are undertaken in AI agents.

Decision-making selects what to do next. Action executes the chosen step.

Example task: "Find the best restaurant near me." The agent decides whether it needs location, cuisine preference, budget, and ratings. It may decide to call a map/search API, filter results by rating and distance, compare options, and present a recommendation. The decision is the selected next step or tool. The action is the actual API call, database query, message, booking request, or output.

Exam key: Decision = choose next step/tool. Action = execute it through tools/APIs or response generation.

Q44. What is the main purpose of the learning phase in the cognitive cycle and how is it achieved?

The learning phase improves future performance. After acting, the agent checks whether the result succeeded, failed, or partially worked. It stores useful information in memory and updates strategies.

Example: if a search tool gave accurate restaurant results, the agent records that tool as effective for local recommendations. If the tool failed, it records the failure and may prefer another tool next time.

Learning is achieved through feedback, outcome evaluation, memory updates, user corrections, success/failure tracking, and sometimes model retraining or fine-tuning. In many agent apps, learning means updating memory and strategy rather than changing the base LLM weights.

Exam key: Learning = feedback -> evaluate outcome -> update memory/knowledge/strategy for better future behavior.

Q45. What is the role of memory in AI agents? How is memory updated and used? Illustrate using a simple example.

Memory gives the agent continuity. It allows the agent to remember user preferences, previous actions, results, facts, and strategies.

Memory can include working memory for the current conversation, short-term summaries, long-term semantic memory for facts, episodic memory for specific past events, and procedural memory for learned strategies.

Example: the user says, "I prefer Italian food and cheap restaurants." Later the user asks, "Recommend a restaurant." The agent retrieves that preference from memory and recommends affordable Italian restaurants instead of asking from zero.

Memory is updated by storing user inputs, tool outputs, final results, feedback, and important facts. It is used by retrieving relevant past context during perception, reasoning, planning, and action.

Exam key: Memory = context + experience. It stores past interactions/results and retrieves them to personalize reasoning and planning.

Q46. What is RAG, and what is the role of RAG in building informed AI agents?

RAG means Retrieval-Augmented Generation. It combines an LLM with external knowledge. The system retrieves relevant information from documents, databases, or vector stores and gives that information to the model before it generates an answer.

For AI agents, RAG is important because agents must often act using current, private, or domain-specific knowledge that is not inside the model's training data. RAG reduces hallucination, improves accuracy, supports private course/company documents, and gives agents grounded evidence for reasoning and decisions.

Exam key: RAG = retrieve external knowledge + generate answer. It makes agents more informed, accurate, and less likely to hallucinate.

Q47. Describe in detail the support provided by LangChain for building autonomous AI agents.

LangChain supports autonomous agents by providing modular building blocks around LLMs.

It provides model wrappers to connect to chat/LLM providers; prompt templates to structure instructions; tools to let agents call APIs, search engines, calculators, databases, and custom functions; agents to decide which tool to use; memory to preserve context; retrievers and vector stores for RAG; chains/runnables for fixed workflows; and LangGraph for stateful graph-based agent loops.

It also supports multi-agent patterns where different agents specialize in roles such as researcher, planner, coder, reviewer, or writer. For example, a researcher agent gathers information, a planner creates steps, and a writer generates the final response. This matches the agentic idea of modular cognitive functions.

Exam key: LangChain supports agents through LLM wrappers, prompts, tools, agents, memory, RAG/retrievers/vector stores, chains, LangGraph, and multi-agent workflows.

Final Double-Check Summary

- All remaining AI tutorial files from the uploaded zip were included: 2.1, 2.2, 2.3, and 2.4.
- All explicit questions and compound subquestions were answered in order.
- Course-specific wording was preserved where useful, but obvious technical issues were corrected with accuracy notes.
- The MLP activation/loss question was answered carefully: sigmoid is explicit in the slides; the loss is described generally as target-vs-output error minimized by gradient descent.
- The Transformer answer distinguishes the original encoder-decoder Transformer from GPT-style decoder-only Transformers.
- The Agentic AI answers include the full cognitive loop: perception, reasoning, planning, action, learning, supported by memory and tools.